

***Adaptation of derivation of William Pugh's Skip List
Algorithm [1 – based indexing] to our program
[0 – based indexing]***

0 – based indexing:

- ***The Ground Floor : Level 0. Every single node has at least 1 pointer.
(0 coin flips needed, Probability = $p^0 = 1$.)***
- ***The Next Floor : Level 1. To reach Level 1, a node has to win exactly 1
coin flip. (1 coin flips needed, Probability = p^1 .)***
- ***The i^{th} Floor : Level i . To reach Level i , a node has to win exactly i
coin flip. (i coin flips needed, Probability = p^i .)***

Or, we may write :

- ***Level L : Required L consecutive coin flips .Probability is p^L .***

For a list of n elements, the expected number of nodes at level L is given by:

$$***$E[\text{nodes at level } L] = n \cdot p^L$***$$

Analysis suggests that ideally we should start a search at the level L where we

expect $\frac{1}{p}$ nodes i.e. if $p = \frac{1}{2}$ means it has 2 nodes on left or right (equally likely events).

$$***$n \cdot p^L = \frac{1}{p}$***$$

$$***$\Rightarrow p^L \times p = \frac{1}{n}$***$$

$$\Rightarrow p^{L+1} = \frac{1}{n}$$

$$\Rightarrow n \times p^{L+1} = 1$$

$$\text{or, roughly } n \times p^L = 1$$

$$\text{or, } p^L = \frac{1}{n}$$

Solving for L:

$$n = \frac{1}{p^L}$$

$$\Rightarrow n = \frac{1}{p^L}$$

$$\Rightarrow n = \left(\frac{1}{p}\right)^L$$

By definition of logarithm [$b^y = x$, then $\log_b(x) = y$]:

$$\Rightarrow L = \log_{\frac{1}{p}}(n)$$

formally William Pugh write it as:

$$\Rightarrow L(n) = \log_{\frac{1}{p}}(n)$$

Because we are using 0 – based indexing, the ideal expected maximum index (the highest floor we expect to exist) is actually one less than the total number of levels, meaning the expected maximum level index is:

$$L(n) - 1.$$

2. Deriving the expected Maximum Level in a List of `n` Elements and Expected Search Cost through probabilistic analysis:

A **random variable** has a fixed but unpredictable value and a predictable probability distribution and average. If X is a random variable, $\text{Prob}\{X = t\}$ denotes the probability that X equals t and $\text{Prob}\{X > t\}$ denotes the probability that X is greater than t .

Example 1:

Translation: "What is the probability that my die roll (X) exactly equals my target number (t)?"

$\text{Prob}\{X = 6\}$ is exactly $\frac{1}{6}$.

Example 2:

A die has 6 possible outcomes: 1, 2, 3, 4, 5, 6. The question asks for the probability of rolling greater than 3. The winning numbers are 4, 5, and 6. That gives you 3 winning outcomes out of 6 total possibilities.

$$\text{Prob}\{X > 3\} = \frac{3}{6} = \frac{1}{2}.$$

Definition:

Pugh sets up three variables:

- X : A random variable representing the time it takes to run Algorithm A_X .
- Y : A random variable representing the time it takes to run Algorithm A_Y .
- t : A specific time limit (for example, 5 milliseconds)

Definition 1: Probabilistic Equality ($=_{\text{prob}}$)

The formal definition is: $X =_{\text{prob}} Y$ if for all t , $\text{Prob}\{X > t\} = \text{Prob}\{Y > t\}$.

- *The Translation: Algorithm X and Algorithm Y are probabilistically identical.*
- *What it means: No matter what time limit (t) we pick, both algorithms have the exact same chance of taking longer than that limit.*
- *Example: If there is a 10% chance that Algorithm X takes longer than 5 milliseconds, there is also exactly a 10% chance that Algorithm Y takes longer than 5 milliseconds. If they match for every possible time limit, they are $=_{\text{prob}}$.*

Definition 2: Probabilistic Inequality ($\leq_{\text{prob}}$)

The formal definition is: $X \leq_{\text{prob}} Y$ if for all t , $\text{Prob}\{X > t\} \leq \text{Prob}\{Y > t\}$.

We define $X \leq_{\text{prob}} Y$ to be true if and only if for any value t , the probability that X exceeds t is less than the probability that Y exceeds t .

- *The Translation: "Algorithm X is probabilistically faster than (or equal to) Algorithm Y."*
- *What it means: Pick any time limit (t). The chance that Algorithm X misses that deadline is always smaller than (or equal to) the chance that Algorithm Y misses that deadline.*
- *The Pizza Analogy: Imagine X and Y are pizza delivery places, and t is 30 minutes. If Place X has a 5% chance of taking longer than 30 minutes, and Place Y has a 20% chance of taking longer than 30 minutes, then Place X is probabilistically faster. If this remains true whether t is 10 minutes, 30 minutes, or 2 hours, then $X \leq_{\text{prob}} Y$.*

Applying the Bounds to the 0 – Based Skip List

Let M be a random variable representing the actual maximum level present in a list of n elements.

Probability that a single node exceeds level k : Because we are using 0 – based indexing, reaching level k requires k coin flips.

To exceed level k (i.e., reach at least level $k + 1$), it requires $k + 1$ coin flips. Therefore, the probability is p^{k+1} .

Probability that no node in a list of n nodes exceeds level k : $(1 - p^{k+1})^n$.

Bernoulli's Inequality : $(1 - x)^n \geq 1 - nx$

Imagine rolling a 100 sided die. The chance of rolling a `1` is 1%

which means $x = 1\% = \frac{1}{100} = 0.01$.

if we roll it 3 times ($n = 3$), what is the chance we never roll a 1?

- **Exact Math**: $(1 - 0.01)^3 = 0.99^3 \approx 0.970299$
- **Shortcut Math**: $1 - (3 \times 0.01) = 1 - 0.03 = 0.97$

Its visible that 0.970299 is strictly greater than 0.97.

Probability that the maximum level M is greater than k :

$$\text{Prob}(M > k) = 1 - (1 - p^{k+1})^n$$

Here, $(1 - p^{k+1})^n$ is the exact chance that the list's height is k or shorter (failure rate), so subtracting it from 1 (100%) gives the probability that $M > k$.

Using Bernoulli's Inequality, where $(1 - x)^n \geq 1 - nx$ for $x \in [0, 1]$, we get $1 - (1 - x)^n \leq nx$. Plugging this in, we get:

$$\text{Prob}(M > k) = 1 - (1 - p^{k+1})^n < n \times p^{k+1}$$

$$\left[\begin{array}{l} \text{As, } (1 - p^{k+1})^n \text{ is strictly greater than} \\ 1 - np^{k+1} \text{ i. e., } (1 - p^{k+1})^n > 1 - np^{k+1} \\ \text{, the inverted formula} \\ \text{becomes strictly less than:} \\ 1 - (1 - p^{k+1})^n < n \times p^{k+1} \end{array} \right]$$

Negative Binomial Distribution and the Overhang:

A binomial distribution is a statistical probability model that counts the number of successes in a fixed number of independent yes or no trials. NB stands for Negative Binomial distribution. Pugh defines $NB(s, p)$ as the number of failures $(1 - p)$ we see while getting our s^{th} success in a sequence of independent Bernoulli trials with success probability p .

- **Average:** $\mathbb{E}[NB(s, p)] = \frac{s(1-p)}{p}$.
- **Variance:** $\text{Var}(NB(s, p)) = \frac{s(1-p)}{p^2}$.

In the formula $NB(1, 1 - p)$:

- **The 1 (The Target):** This is the s in the formula, meaning we are waiting for exactly 1 success.
- **$1 - p$:** This is the probability of success. Therefore:

$$\mathbb{E}[NB(1, 1 - p)] = \frac{1 \times (1 - (1 - p))}{1 - p} = \frac{p}{1 - p}$$

- **$1 - (1 - p)$ is the probability of failure.**

The Test Subject k (0 – Based Adjustment)

k is an arbitrary variable used to test boundaries. We want to test a level k that sits exactly i floors above our expected target.

For 0 – based indexing, the ideal target maximum index is $L(n) - 1$. Therefore, we set our test level k to:

$$k = (L(n) - 1) + i$$

Substituting this into our probabilistic upper bound:

$$\begin{aligned} \text{Prob}(M > k) &< n \times p^{k+1} \\ \Rightarrow \text{Prob}(M > (L(n) - 1) + i) &< n \times p^{(L(n)-1+i)+1} \\ \Rightarrow \text{Prob}(M > (L(n) - 1) + i) &< n \times p^{L(n)-1+1+i} \\ \Rightarrow \text{Prob}(M > L(n) - 1 + i) &< n \times p^{L(n)+i} \end{aligned}$$

Now, expand the exponents:

$$n \times p^{L(n)+i} = n \times p^{L(n)} \times p^i$$

We know $L(n) = \log_{\frac{1}{p}}(n)$. By taking the reciprocal, we know that

$p^{L(n)} = \frac{1}{n}$. Plugging this in:

$$n \times p^{L(n)} \times p^i = n \times \frac{1}{n} \times p^i = p^i$$

So, we have proven:

$$\text{Prob}(M > L(n) - 1 + i) < p^i$$

Pugh also observes that $\text{Prob}(NB(1, 1 - p) + 1 > i) = p^i$.

$NB(1, 1 - p) + 1$ represents the random extra number of levels by which the maximum node level can extend beyond the target, corresponding to an unusually long sequence of successful promotions.

Because both probabilities equal p^i , we can establish a formal probabilistic upper bound for M :

$$M \leq_{\text{prob}} (L(n) - 1) + NB(1, 1 - p) + 1$$

$$\therefore L(n) - 1 + NB(1, 1 - p) + 1$$

$$\Rightarrow L(n) + NB(1, 1 - p) - 1 + 1$$

$$\Rightarrow L(n) + NB(1, 1 - p)$$

$$\therefore M \leq_{\text{prob}} L(n) + NB(1, 1 - p)$$

Translated into plain English: The maximum height of the entire skip list (M) will safely be less than or equal to our ideal target height, plus a tiny, rapidly shrinking statistical streak of extra coin flips.

Taking the expected value of M (the average of M):

We know,

$$\mathbb{E}[NB(1, 1 - p)] = \frac{p}{1 - p}$$

$$\therefore \mathbb{E}[L(n) + NB(1, 1 - p)] = L(n) + \frac{p}{1 - p}$$

$$\text{Hence, } E[M] \leq L(n) + \frac{p}{1 - p}$$

4. Expected Search Cost

To find the expected search cost, we trace the search path backward (up and to the left) and sum the expected movements.

- *Upward movements from Level 0 to $L(n) - 1$:*

This requires exactly $L(n) - 1$ upward movements.

- *Leftward movements below $L(n) - 1$:*

For every upward move, the expected number of leftward moves is governed by $NB(L(n) - 1, p)$.

$$\mathbb{E}[NB(L(n) - 1, p)] = \frac{(L(n) - 1)(1 - p)}{p}$$

Summing the cost to climb to the target level:

$$\begin{aligned}\mathbb{E}[(L(n) - 1) + NB(L(n) - 1, p)] &= (L(n) - 1) + \frac{(L(n) - 1)(1 - p)}{p} \\ &= (L(n) - 1) \left(1 + \frac{1 - p}{p}\right) \\ &= (L(n) - 1) \left(\frac{p + 1 - p}{p}\right) \\ &= \frac{L(n) - 1}{p}\end{aligned}$$

Leftward movements remaining once at $L(n) - 1$:

Bounded by the number of nodes with a level $\geq L(n) - 1$.

This follows a Binomial distribution $B\left(n, \frac{1}{np}\right)$.

$$\mathbb{E}\left[B\left(n, \frac{1}{np}\right)\right] = n \times \frac{1}{np} = \frac{1}{p}$$

Extra upward movements from $L(n) - 1$ to the maximum level:

As derived earlier, this distance is $NB(1, 1 - p)$ for 0-based indexing.

$$\mathbb{E}[NB(1, 1 - p)] = \frac{p}{1 - p}$$

Total Expected Search Cost:

Taking expectations gives:

$$\mathbb{E}[\text{search cost}] = \frac{L(n) - 1}{p} + \frac{1}{p} + \frac{p}{1 - p}$$

Simplify the equation:

$$\Rightarrow \mathbb{E}[\text{search cost}] = \frac{L(n) - 1 + 1}{p} + \frac{p}{1 - p}$$

$$\Rightarrow \mathbb{E}[\text{search cost}] = \frac{L(n)}{p} + \frac{p}{1 - p}$$

Plugging in $p = 0.5$ (or $\frac{1}{2}$), we get:

$$\begin{aligned} &= \frac{\log_2(n)}{0.5} + \frac{0.5}{1 - 0.5} \\ &= 2 \times \log_2(n) + \frac{0.5}{0.5} \\ &= 2 \times \log_2(n) + 1 \end{aligned}$$

Because $0 < p < 1$ and the fractional terms resolve to constants, we can conclude that the expected search cost drops the constants and becomes strictly:

$$= O(\log_2 n)$$

Note:

- $\frac{1}{p}$ is a constant. If $p = 0.5$, this is forever exactly 2.
- $\frac{p}{1-p}$ is a constant. If $p = 0.5$, this becomes $\frac{0.5}{0.5}$, which is forever exactly 1.

$$E[\text{search cost}] = \frac{1}{p} \times L(n) + \frac{p}{1-p}$$

$$E[\text{search cost}] = \frac{1}{p} \times \log_{\frac{1}{p}}(n) + \frac{p}{1-p}$$

*Hence, final search cost := **$O(\log n)$***